

Problem statement

Theme : User generated content

Problem statement name : Rewarding novelty in submissions

Overview

You are faced with judging the “novelty” of some user generated text-based content submitted in response to some larger piece of content. How would you create a pipeline that rewards truly novel content on a normalized scale of [0.0,1.0]?

Tech stack

- Python, Typescript, or Ruby preferred

Task

1. Determine the particular “shape” of user generated content you would evaluate. “User generated content” means structured input a user might submit to an app or website in response to some fixed piece of content e.g., commentary on a news article. Note: The structured content must contain three discrete properties provided by the user (e.g. a headline, body, and a multi-choice selection); the fixed content that the user content is submitted against should be 100 words or less.
2. Describe how you would reward submitted user generated content based on its novelty relative to other submissions of its kind, while retaining a minimum level of relevancy to the fixed content in question.
3. Implement a mechanism to evaluate the novelty of a newly submitted piece of content against ≈ 50 other pieces of content. Note: synthetic generation of such content via an LLM is recommended.
4. Demonstrate how you effectively reward truly novel content, do not reward non-”novel” content, and remain relevant to the fixed content through automated tests. You must show high novelty, but low relevance submissions are not rewarded.

Restrictions

- A coding agent may be used, but you must explain how you collaborated with (prompted) the agent to arrive at your solution.

Submission format

- Details about your solution, including an explanation of your engineering design, your rationale for it, definition of success criteria for your solution, and level of achievement against your criteria, in a Markdown or PDF file
- Disclosure of coding agent use: You must disclose the use of coding agents in developing your project. Where used, you must explain how you directed the agent to arrive at the outputs it generated. This may be achieved by sharing documentation of the architectural requirements you supplied to the agent, or by providing summarized traces of your interactions with the agent.
- GitHub/GitLab for code commit containing your solution, as well as your golden dataset and tests